

What is NoSQL?

Spin up a NoSQL Cluster Free

Try a NoSQL Database with Atlas >

What is NoSQL?

NoSQL databases (AKA "not only SQL") store data differently than relational tables. NoSQL databases come in a variety of types based on their data model. The main types are [document](#), key-value, wide-column, and graph. They provide flexible schemas and scale easily with large amounts of big data and high user loads.

In this article, you'll learn what a NoSQL database is, *why* (and when!) you should use one, and *how* to get started.

Table of contents

- [What is a NoSQL Database?](#)
- [Types of NoSQL Database](#)
- [Brief History of NoSQL Databases](#)
- [NoSQL Database Features](#)
- [Relational database vs NoSQL database example](#)

- [Differences between RDBMS and NoSQL databases](#)
- [NoSQL use cases](#)
- [When should NoSQL be Used?](#)
- [NoSQL Database Misconceptions](#)
- [NoSQL Query Tutorial](#)
- [Summary](#)
- [FAQs](#)

What is a NoSQL database?

When people use the term “NoSQL database,” they typically use it to refer to any non-relational database. Some say the term “NoSQL” stands for “non-SQL” while others say it stands for “not only SQL.” Either way, most agree that NoSQL databases store data in a more natural and flexible way. NoSQL, as opposed to SQL, is a database management approach, whereas SQL is just a query language, similar to the query languages of NoSQL databases.

Types of databases – NoSQL

Over time, four major [types of NoSQL databases](#) have emerged: document databases, [key-value databases](#), wide-column stores, and graph databases. Nowadays, multi-model databases are also becoming quite popular.

Document-oriented databases

A document-oriented database stores data in documents similar to JSON (JavaScript Object Notation) objects. Each document contains pairs of fields and values. The values can typically be a variety of types, including things like strings, numbers, booleans, arrays, or even other objects. A document database offers a flexible data model, much suited for semi-structured and typically unstructured data sets. They

also support nested structures, making it easy to represent complex relationships or hierarchical data.

Examples of document databases are [MongoDB](#) and Couchbase. A typical document will look like the following:

```
1  {
2    "_id": "12345",
3    "name": "foo bar",
4    "email": "foo@bar.com",
5    "address": {
6      "street": "123 foo street",
7      "city": "some city",
8      "state": "some state",
9      "zip": "123456"
10   },
11   "hobbies": ["music", "guitar", "reading"]
12 }
```

Key-value databases

A key-value store is a simpler type of database where each item contains keys and values. Each key is unique and associated with a single value. They are used for caching and session management and provide high performance in reads and writes because they tend to store things in memory. Examples are Amazon DynamoDB and Redis. A simple view of data stored in a key-value database is given below:

```
1 Key: user:12345
2 Value: {"name": "foo bar", "email": "foo@bar.com",
  "designation": "software developer"}
```

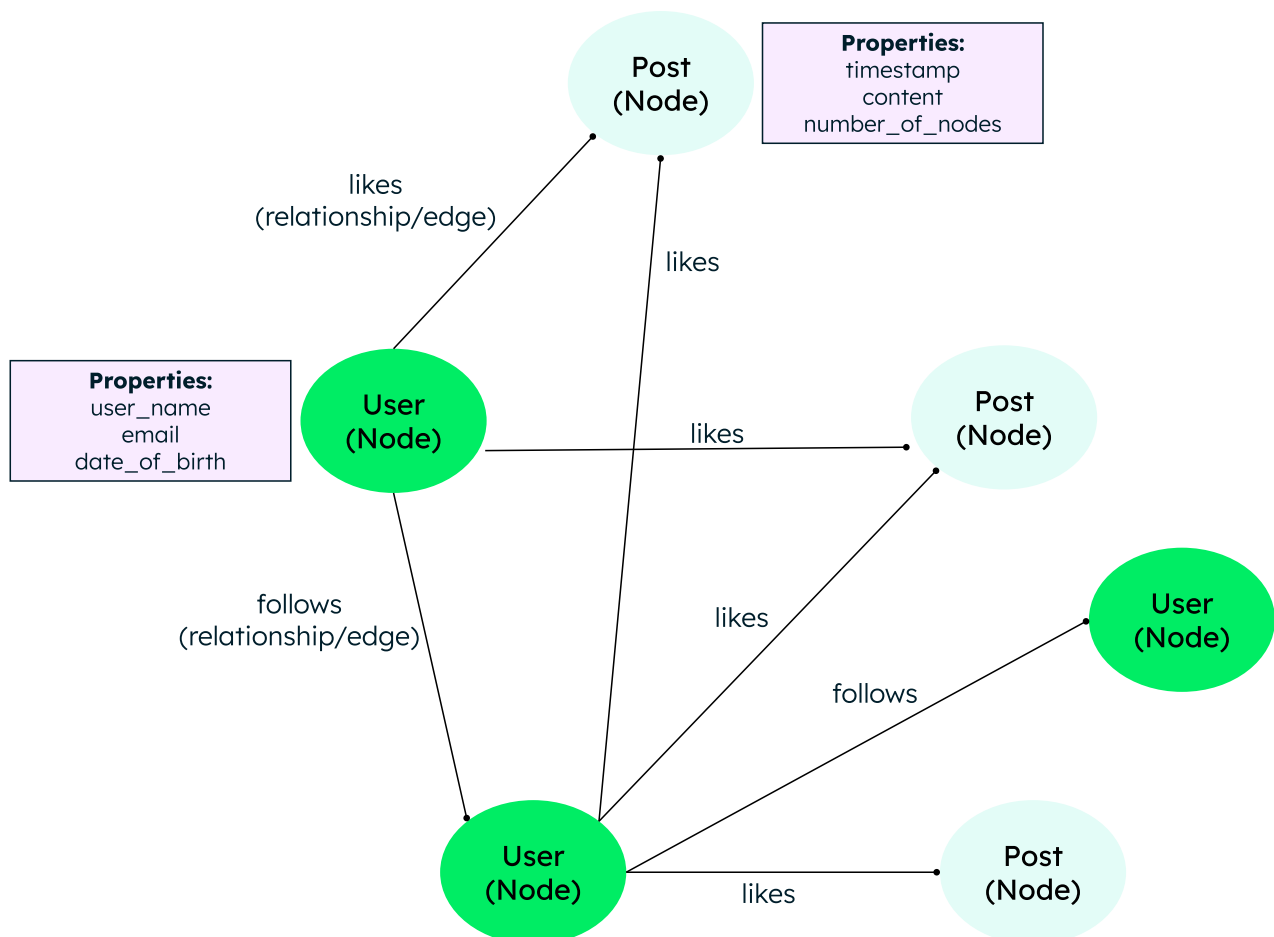
Wide-column stores

Wide-column stores store data in tables, rows, and dynamic columns. The data is stored in tables. However, unlike traditional SQL databases, wide-column stores are flexible, where different rows can have different sets of columns. These databases can employ column compression techniques to reduce the storage space and enhance performance. The wide rows and columns enable efficient retrieval of sparse and wide data. Some examples of wide-column stores are Apache Cassandra and HBase. A typical example of how data is stored in a wide-column is as follows:

name	id	email	dob	city
Foo bar	12345	foo@bar.com		Some city
Carn Yale	34521	bar@foo.co m	12-05-1972	

Graph databases

A graph database stores data in the form of nodes and edges. Nodes typically store information about people, places, and things (like nouns), while edges store information about the relationships between the nodes. They work well for highly connected data, where the relationships or patterns may not be very obvious initially. Examples of graph databases are Neo4J and Amazon Neptune. MongoDB also [provides graph traversal capabilities](#) using the \$graphLookup stage of the aggregation pipeline. Below is an example of how data is stored:

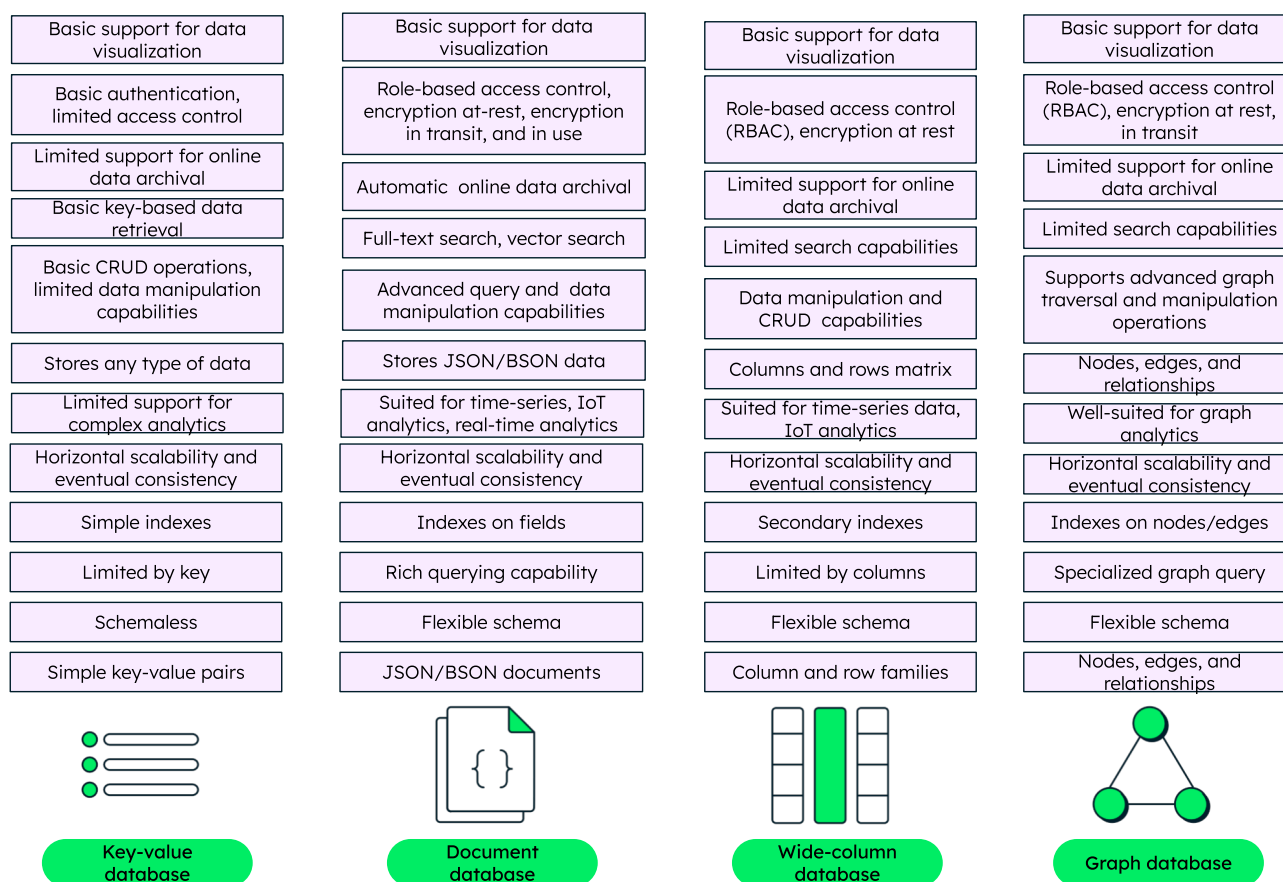


Multi-model databases

Multi-model databases support more than one type of NoSQL data model so that developers can choose based on their application requirements. These databases have a unified database engine that can handle multiple data models within a database instance. Examples are CosmosDB and ArangoDB.

Quick comparison of types of databases – NoSQL

Each of the NoSQL databases offers different features. For example, graph databases could be more suited for analyzing complex relationships and patterns between entities, while document databases provide a more flexible, natural way of storing and retrieving large data volumes of similar types as documents. The choice of database depends on the use case you want to develop.

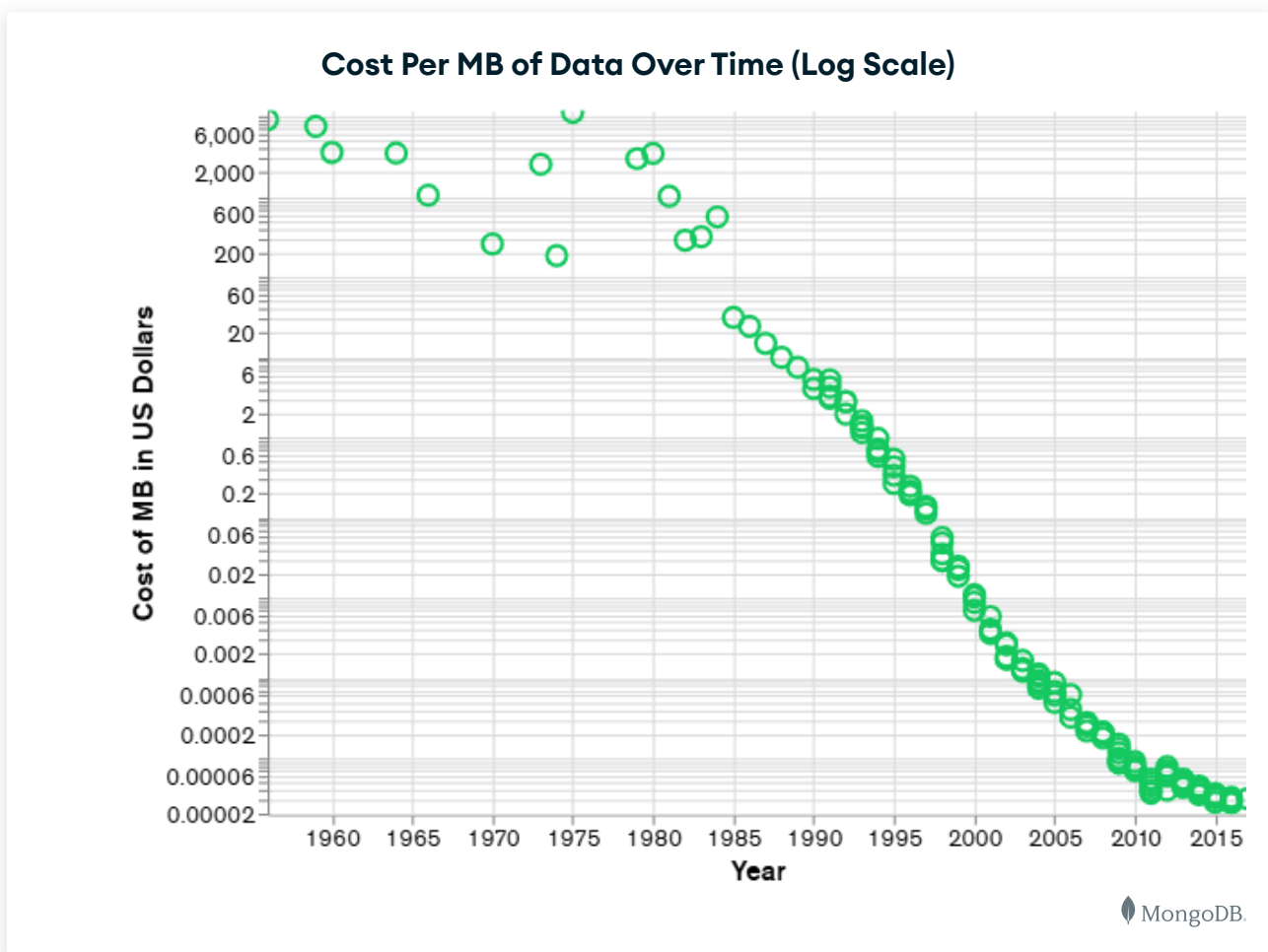


To learn more, read

[Understanding the Different Types of NoSQL Databases.](https://www.mongodb.com/resources/basics/databases/nosql-explained)

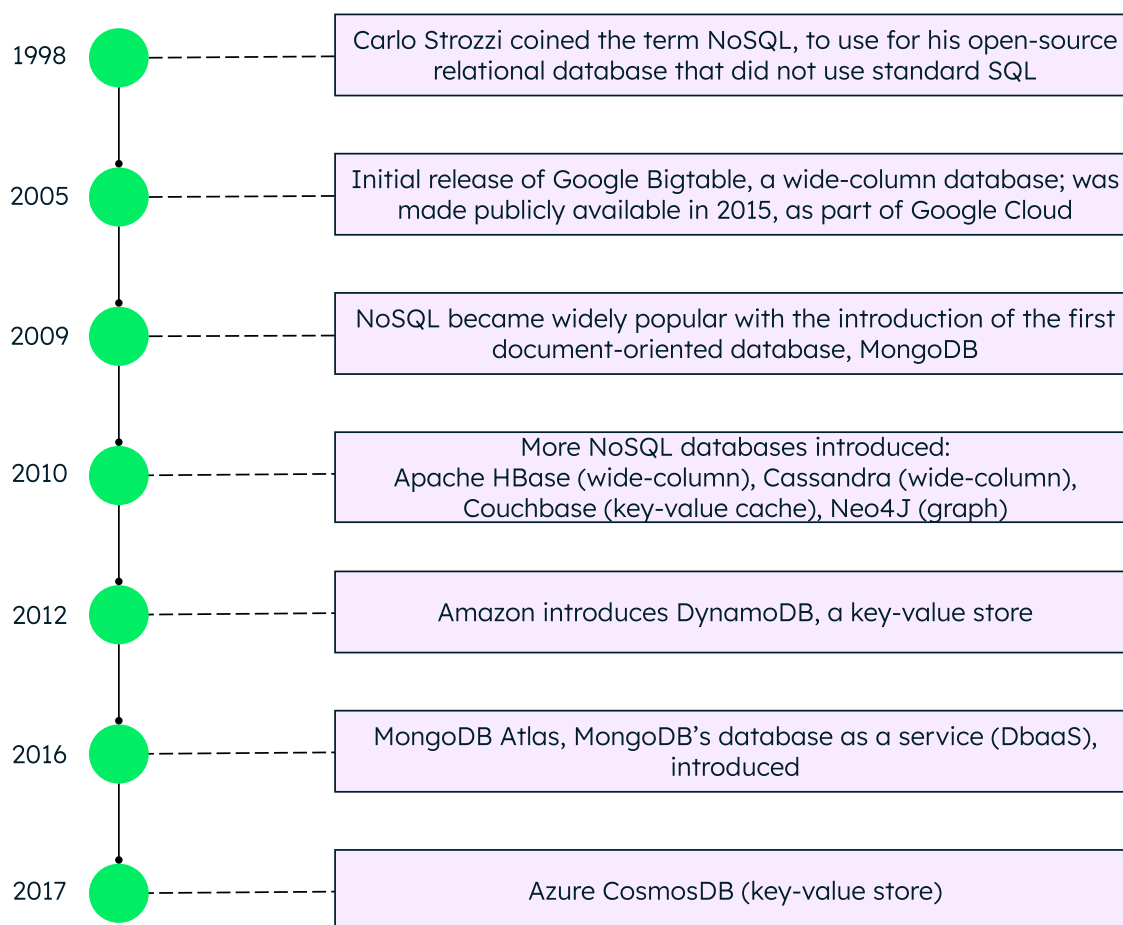
Brief history of NoSQL databases

NoSQL databases emerged in the late 2000s as the cost of storage dramatically decreased. Gone were the days of needing to create a complex, difficult-to-manage data model in order to avoid data duplication. NoSQL databases optimized for developer productivity.



As storage costs rapidly decreased, the amount of data that applications needed to store and query increased. This data came in all shapes and sizes – **structured, semi-structured, and unstructured** –

and defining the schema in advance became nearly impossible. NoSQL databases allow developers to store huge amounts of unstructured data, giving them a lot of flexibility.



In the early 2000s, a paper published by Google on BigTable, the wide-column database, explored the wide range of possibilities for a distributed storage system. 2009 saw a major rise in NoSQL databases, with two key document-oriented databases, MongoDB and CouchDB, coming into the picture.

By the 2010s, different types of NoSQL databases emerged and the acceptance of NoSQL became widespread, with businesses becoming more data-driven.

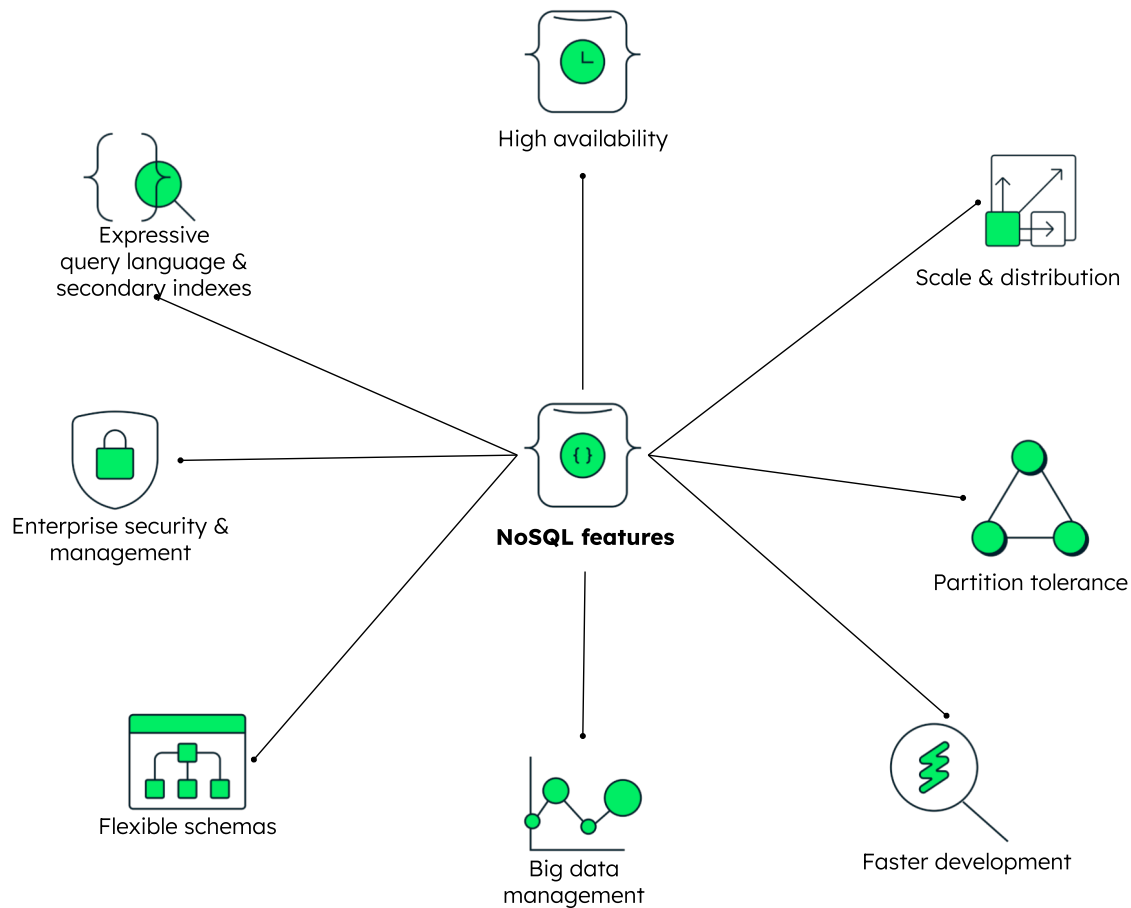
Additionally, the Agile Manifesto was rising in popularity, and software engineers were rethinking the way they developed software. They had to rapidly adapt to changing requirements, iterate quickly, and make changes throughout their software stack – all the way down to the database. NoSQL databases gave them this flexibility.

Cloud computing also rose in popularity, and developers began using public clouds to host their applications and data. They wanted the ability to distribute data across multiple servers and regions to make their applications resilient, to scale out instead of scale up, and to intelligently geo-place their data. Some NoSQL databases, like [MongoDB Atlas](#), provide these capabilities.

Due to the exponential growth of digitization, businesses now collect as much unstructured data as possible. To be able to analyze and derive actionable [real-time insights](#) from such [big data](#), businesses need modern solutions that go beyond simple storage. Businesses need a platform that can easily [scale](#), [transform](#), and [visualize data](#); create dashboards, reports, and [charts](#); and work with AI and [business intelligence tools](#) to accelerate their business productivity. Due to their flexible and distributed nature, NoSQL databases (for example, MongoDB) shine in these tasks.

NoSQL database features

NoSQL databases are flexible, scalable, and distributed databases. Different types of NoSQL databases have their own unique features.



At a high level, NoSQL databases typically have the following features:

- **Distributed computing**
- **Scaling**
- **Flexible schemas and rich query language**
- **Ease of use for developers**
- **Partition tolerance**
- **High availability**

BASE compliance

NoSQL databases are BASE compliant, i.e., basic availability soft state eventual consistency. Basic availability refers to the ability of the system to tolerate a partial failure (like a loss of a node). Soft state means that the system allows temporary inconsistencies before eventually achieving consistency automatically over time. BASE compliance ensures high availability, faster data processing,

scalability, and flexibility. However, MongoDB can also be configured to provide [multi-document ACID compliance](#).

Learn more about [the advantages of NoSQL databases](#).

Relational database vs NoSQL database example

Let's consider an example of storing information about a user and their hobbies. We need to store a user's first name, last name, cell phone number, city, and hobbies.

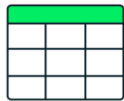
In a relational database management system (RDBMS), we'd likely create two tables: one for Users and one for Hobbies.

In order to retrieve all of the information about a user and their hobbies, information from the Users table and Hobbies table will need to be joined together.

The data model we design for a NoSQL database will depend on the type of NoSQL database we choose. Let's consider how to store the same information about a user and their hobbies in a [document database](#) like MongoDB.

In order to retrieve all of the information about a user and their hobbies, a single document can be retrieved from the database. No joins are required, resulting in faster queries.

RDBMS vs NoSQL (Document)



Relational Database

User table

ID	first_name	last_name	cell	city
1	Leslie	Yepp	8125552344	Pawnee

Hobbies table

ID	user_id	hobby
10	1	scrapbooking
11	1	eating waffles
12	1	working



MongoDB

```
{
  "_id": 1,
  "first_name": "Leslie",
  "last_name": "Yepp",
  "cell": "8125552344",
  "city": "Pawnee",
  "hobbies": ["scrapbooking", "eating waffles", "working"]
}
```

- No need for joins
- No need for data normalization

To see a more detailed version of this data modeling example, read [Mapping Terms and Concepts From SQL to MongoDB](#).

Differences between RDBMS and NoSQL databases

There are a variety of differences between relational database management systems and non-relational databases. One of the key differences is the way data is modeled in the database. Some key differences of each feature is listed below:

Data modeling

NoSQL: Data models vary based on the type of NoSQL database used – for example, key-value, document, graph, and wide-column – making the model suitable for semi-structured and unstructured data.

RDBMS: RDBMS uses a tabular data structure, with data represented as a set of rows and columns, making the model suitable for structured

data.

Schema

NoSQL: It provides a flexible schema where each set of documents/row-column/key-value pairs can contain different types of data. It's easier to change schema, if required, due to the flexibility.

RDBMS: This is a fixed schema where every row should contain the same predefined column types. It is difficult to change the schema once data is stored.

Query language

NoSQL: It varies based on the type of NoSQL database used. For example, MongoDB has **MQL**, and Neo4J uses Cypher.

RDBMS: This uses structured query language (SQL).

Scalability

NoSQL: NoSQL is designed for vertical and horizontal scaling.

RDBMS: RDBMS is designed for vertical scaling. However, it can extend limited capabilities for horizontal scaling.

Data relationships

NoSQL: Relationships can be nested, explicit, or implicit.

RDBMS: Relationships are defined through foreign keys and accessed using joins.

Transaction type

NoSQL: Transactions are either **ACID**- or BASE-compliant.

RDBMS: Transactions are ACID-compliant.

Performance

NoSQL: NoSQL is suitable for real-time processing, big data analytics, and distributed environments.

RDBMS: RDBMS is suitable for read-heavy and transaction workloads.

Data consistency

NoSQL: This offers eventual consistency, in most cases.

RDBMS: This offers high data consistency.

Distributed computing

NoSQL: One of the main reasons to introduce NoSQL was for distributed computing, and NoSQL databases support distributed data storage, vertical and horizontal scaling through sharding, replication, and clustering.

RDBMS: RDBMS supports distributed computing through clustering and replication. However, it's less scalable and flexible as it's not traditionally designed to support distributed architecture.

Fault tolerance

NoSQL: NoSQL has built-in fault tolerance and high availability due to data replication.

RDBMS: RDBMS uses replication, backup, and recovery mechanisms. However, as they are designed for these, additional measures like disaster recovery mechanisms may need to be implemented during application development.

Data partitioning

NoSQL: It's done through sharding and replication.

RDBMS: It supports table-based partitioning and partition pruning.

Learn more about [data partitioning here](#).

Data to object mapping

NoSQL: NoSQL stores the data in a variety of ways – for example, as JSON documents, wide-column stores, or key-value pairs. It provides abstraction through the ODM (object-data mapping) frameworks to work with NoSQL data in an object-oriented manner.

RDBMS: RDBMS relies more on data-to-object mapping so that there is seamless integration between the database columns and the object-oriented application code.

To learn more about the differences between relational databases and NoSQL databases, read [NoSQL vs SQL Databases](#).

NoSQL use cases

NoSQL database systems are used in nearly every [industry](#), for real-time analytics, content management, IoT applications, recommendation systems, fraud detection, product catalog management, and much more. Use cases range from the highly critical

(e.g., storing [financial data](#) and [healthcare records](#)) to the more fun and frivolous (e.g., [storing IoT readings from a smart kitty litter box](#)).

When should NoSQL be used?

When deciding which database to use, decision-makers typically find one or more of the following factors that lead them to select a NoSQL database:

- Fast-paced Agile development
- Storage of structured and semi-structured data
- Huge volumes of data
- Requirements for scale-out architecture
- Modern application paradigms like microservices and real-time streaming

See [When to Use NoSQL Databases](#) and [Exploring NoSQL Database Examples](#) for more detailed information on the reasons listed above.

NoSQL database misconceptions

Over the years, many misconceptions about NoSQL databases have spread throughout the developer community. In this section, we'll discuss two of the most common misconceptions.

Misconception: relationship data is best suited for relational databases

A common misconception is that NoSQL databases or non-relational databases don't store relationship data well. NoSQL databases can store relationship data – they just store it differently than relational databases do.

In fact, [when compared with relational databases](#), many find modeling relationship data in NoSQL databases to be easier than in relational databases because related data doesn't have to be split between tables. NoSQL data models allow related data to be nested within a single data structure.

Misconception: NoSQL databases don't support ACID transactions

Another common misconception is that NoSQL databases don't support ACID transactions. Some NoSQL databases, like MongoDB, do, in fact, support [ACID transactions](#).

Note that the way data is modeled in NoSQL databases can eliminate the need for multi-record transactions in many use cases. Consider the earlier example where we stored information about a user and their hobbies in both a relational model and a document store. To ensure information about a user and their hobbies was updated together in a relational database, we'd need to use a transaction to update records in two tables. To do the same in a document store, we could update a single document – no multi-record transaction required.

To learn more about common misconceptions, read [Everything You Know About MongoDB is Wrong](#).

NoSQL query tutorial

You could start with MongoDB, the world's most popular NoSQL database, according to DB-Engines. The easiest way to get started with MongoDB is [MongoDB Atlas](#). Atlas is MongoDB's fully managed database as a service. Atlas has a forever-free tier, which you can use to play around. Check out the [MongoDB Atlas tutorial](#) to get started.

You can continue to interact with your data by using the Atlas Data Explorer to insert new documents, edit existing documents, and delete documents.

When you are ready to try more advanced queries that aggregate your data, [create an aggregation pipeline](#). The aggregation framework is an incredibly powerful tool for analyzing your data. To learn more, take the free MongoDB University Course [M121 The MongoDB Aggregation Framework](#).

When you want to visualize your data, check out [MongoDB Charts](#). Charts allow you to create dashboards that are filled with visualizations of your data.

Summary

NoSQL databases provide a variety of benefits, including flexible data models, horizontal scaling, lightning-fast queries, and ease of use for developers. NoSQL databases come in a variety of types, including document stores, key-values databases, wide-column stores, graph databases, and multi-model databases.

MongoDB is the world's most popular NoSQL database.
[Learn more about MongoDB Atlas](#), and give the free tier a try.

Excited to learn more now that you have your own Atlas account? Head over to [MongoDB University](#) where you can get free online training from MongoDB engineers and earn a [MongoDB certification](#). The [Quickstarts](#) are another great place to begin; they will get you up and running quickly with your favorite programming language.

FAQs

What are the advantages of NoSQL?



What is eventual consistency?



What is the CAP theorem?



What is NoSQL used for?



What is a NoSQL database?



How do I write a NoSQL query?



Is NoSQL hard to learn?



Is JSON a NoSQL?



What language is used to query NoSQL?



Does NoSQL have schema?



This article was written by [Lauren Schaefer](#), a MongoDB Developer Advocate.

Learn more about key differences between [NoSQL vs SQL Databases](#)

Related NoSQL Resources

- [What are the main differences between NoSQL and SQL?](#)
- [When should you use a NoSQL database?](#)
- [What are the 4 different types of NoSQL databases?](#)
- [NoSQL Databases Advantages](#)
- [NoSQL data modeling and schema design](#)
- [NoSQL Database Examples](#)

Learn More

- [MongoDB Compatibility](#)
- [MongoDB Basics](#)
- [Learn About Databases](#)

- [Languages compatible with MongoDB](#)

Follow this tutorial with MongoDB Atlas

Experience the benefits of using MongoDB, the premier NoSQL database, on the cloud.

Get Started Free!

About

Careers

Investor Relations

Legal

GitHub

Security Information

Trust Center

Connect with Us

Support

Contact Us

Customer Portal

Atlas Status

Customer Support

Manage Cookies

Deployment Options

MongoDB Atlas

Enterprise Advanced

Community Edition

Data Basics

Vector Databases

NoSQL Databases

Document Databases

RAG Database

ACID Transactions

MERN Stack

MEAN Stack

© 2025 MongoDB, Inc.